## 1. Ponteiros: A Função "Trocar"

Esta é a função mais fundamental que usa ponteiros. Ela é a base para todos os algoritmos de ordenação. Se você precisar modificar uma variável de main dentro de uma função, você usará um ponteiro.

**Cenário de Prova:** "Ordene um vetor", "Crie uma função que modifique duas variáveis".

Generated c

```
    /**
 * @brief Troca os valores de duas variáveis inteiras.
 *        Usa ponteiros para alterar os valores nos endereços de memória originais.
 *
 * @param a Ponteiro para a primeira variável.
 * @param b Ponteiro para a segunda variável.
 */
void trocar(int *a, int *b) {
    int temp = *a; // Pega o valor apontado por 'a'
    *a = *b;       // O valor apontado por 'a' recebe o valor apontado por 'b'
    *b = temp;     // O valor apontado por 'b' recebe o valor temporário
}

// Exemplo de como chamar em main():
// int x = 10, y = 20;
// trocar(&x, &y); // Passamos o ENDEREÇO das variáveis
// printf("x = %d, y = %d\n", x, y); // Saída: x = 20, y = 10
```

---

## 2. Structs: Agrupando Dados

Structs são perfeitas para quando o problema lida com uma "coisa" que tem várias características (um aluno, um produto, um livro).

**Cenário de Prova:** "Cadastre 5 produtos com código, nome e preço...", "Leia os dados de N alunos e encontre o que tem a maior nota...".

Generated c

```
    #include <string.h> // Para strcpy

// Define a estrutura (o "molde")
struct Produto {
    int codigo;
    char nome[50];
    float preco;
};

/**
```

```
 * @brief Imprime os dados de uma variável do tipo Produto.
 * @param p A variável struct a ser impressa.
 */
void imprimir_produto(struct Produto p) {
    printf("Codigo: %d\n", p.codigo);
    printf("Nome: %s\n", p.nome);
    printf("Preco: R$ %.2f\n", p.preco);
}

// Exemplo de como usar em main():
// struct Produto prod1; // Declara uma variável do tipo Produto
//
// // Preenche os dados
// prod1.codigo = 101;
// strcpy(prod1.nome, "Caneta Azul"); // Para strings, use strcpy
// prod1.preco = 1.99;
//
// imprimir_produto(prod1);

// DICA: É muito comum usar um VETOR de structs:
// struct Produto estoque[10];
// estoque[0].codigo = 101;
// ...
```

---

## 3. Ordenação: Bolha, Seleção e Inserção

Você precisa ordenar um vetor. Escolha um dos três, o que você achar mais fácil de lembrar. **Bubble Sort (Bolha)** costuma ser o mais intuitivo para escrever sob pressão.

**Cenário de Prova:** "Liste os números em ordem crescente", "Apresente os produtos do mais barato para o mais caro".

**a) Bubble Sort (Ordenação por Bolha)**

Generated c

```
    // Precisa da função trocar() definida acima!
void bubble_sort(int vetor[], int tamanho) {
    for (int i = 0; i < tamanho - 1; i++) {
        for (int j = 0; j < tamanho - i - 1; j++) {
            // Compara elementos adjacentes
            if (vetor[j] > vetor[j + 1]) {
                trocar(&vetor[j], &vetor[j + 1]);
            }
        }
    }
}
```

**b) Selection Sort (Ordenação por Seleção)**

Generated c
```
    // Precisa da função trocar() definida acima!
void selection_sort(int vetor[], int tamanho) {
   for (int i = 0; i < tamanho - 1; i++) {
      int indice_menor = i;
      // Encontra o menor elemento no resto do vetor
      for (int j = i + 1; j < tamanho; j++) {
         if (vetor[j] < vetor[indice_menor]) {
            indice_menor = j;
         }
      }
      // Troca com o elemento da posição atual
      trocar(&vetor[i], &vetor[indice_menor]);
   }
}
```

**c) Insertion Sort (Ordenação por Inserção)**

Generated c
```
    void insertion_sort(int vetor[], int tamanho) {
   for (int i = 1; i < tamanho; i++) {
      int chave = vetor[i];
      int j = i - 1;
      // Move os elementos maiores que a chave para a direita
      while (j >= 0 && vetor[j] > chave) {
         vetor[j + 1] = vetor[j];
         j = j - 1;
      }
      vetor[j + 1] = chave;
   }
}
```

---

## 4. Pesquisa: Sequencial e Binária

Se precisar encontrar um item em um vetor.

**Cenário de Prova:** "Verifique se o número X existe na lista", "Encontre o produto com o código Y".

**a) Pesquisa Sequencial (Funciona em QUALQUER vetor)**

Generated c
```
    /**
 * @brief Procura por um valor em um vetor de forma sequencial.
 * @return O índice do elemento se encontrado, ou -1 se não encontrado.
 */
int pesquisa_sequencial(int vetor[], int tamanho, int valor_procurado) {
   for (int i = 0; i < tamanho; i++) {
      if (vetor[i] == valor_procurado) {
```

```
            return i; // Encontrado! Retorna a posição.
        }
    }
    return -1; // Não encontrado.
}
```

**b) Pesquisa Binária (SÓ FUNCIONA EM VETOR ORDENADO)**

**Dica:** Use esta se o problema garantir que o vetor está ordenado ou se você mesmo o ordenou antes.

Generated c

```
    /**
 * @brief Procura por um valor em um vetor ORDENADO usando pesquisa binária.
 * @return O índice do elemento se encontrado, ou -1 se não encontrado.
 */
int pesquisa_binaria(int vetor[], int tamanho, int valor_procurado) {
    int inicio = 0;
    int fim = tamanho - 1;

    while (inicio <= fim) {
        int meio = inicio + (fim - inicio) / 2; // Evita overflow para números grandes

        if (vetor[meio] == valor_procurado) {
            return meio; // Encontrado!
        }
        if (vetor[meio] < valor_procurado) {
            inicio = meio + 1; // Procura na metade direita
        } else {
            fim = meio - 1; // Procura na metade esquerda
        }
    }
    return -1; // Não encontrado
}
```

---

## 5. Recursividade: A Função que se Chama

Problemas recursivos envolvem quebrar um problema grande em uma versão menor do mesmo problema. Fatorial é o exemplo clássico e mais seguro de implementar.

**Cenário de Prova:** "Calcule o fatorial de um número", "Calcule a soma dos N primeiros números usando recursividade".

Generated c

```
    /**
 * @brief Calcula o fatorial de um número usando recursividade.
 * @param n O número para calcular o fatorial (deve ser não-negativo).
 */
```

```
long long int fatorial(int n) {
    // Caso Base: A condição de parada da recursão.
    if (n == 0 || n == 1) {
        return 1;
    }
    // Passo Recursivo: A função se chama com um problema menor.
    else {
        return n * fatorial(n - 1);
    }
}

// Exemplo de como chamar em main():
// int num = 5;
// printf("O fatorial de %d e %lld\n", num, fatorial(num)); // Saída: 120
```

## Dicas Finais para a Prova

1. **Leia com Calma:** Entenda exatamente o que o problema pede. Qual é a entrada? Qual é a saída esperada?
2. **Identifique a Ferramenta:**
    - Precisa ordenar? -> Use uma das 3 funções de ordenação.
    - Precisa guardar dados de "coisas" (aluno, produto)? -> Use struct.
    - Precisa encontrar algo? -> Se estiver ordenado, pesquisa_binaria. Senão, pesquisa_sequencial.
    - O problema se resolve com uma versão menor dele mesmo? -> recursividade.
    - Precisa modificar algo que veio de main? -> Use ponteiros (trocar é o exemplo perfeito).
3. 
4. **Monte o Quebra-Cabeça:** Sua função main vai orquestrar a chamada dessas funções coringa para resolver o problema.

Boa sorte na prova! Com esses "blocos de montar" na cabeça, você estará muito bem preparado.